

EECE 5639 Project 1: Motion Detection Using Image Filtering

Nathan Foster
Northeastern University
Boston, MA
foster.nat@northeastern.edu

Lucas Mitchell
Northeastern University
Boston, MA
mitchell.lu@northeastern.edu

Abstract— In this report, we implement two types of motion detection algorithms to evaluate their performance and determine what parameters provide the best results. First, we apply a 2D spatial smoothing filter to reduce pixel noise before applying a simple temporal derivative filter to detect motion. The second option is a 1D derivative of gaussian filter, which smooths the frame by temporally adjacent pixels instead of spatially. It was found that utilization of a 1D Derivative of Gaussian (DoG) filter was the most effective method at reducing noise, at the cost of some ghosting artifacts at higher S_σ values. Different derivative thresholds for mask application are also considered: A fixed value of 5 working well for the selected dataset, and a dynamic value of 3 times the standard deviation of the frame providing the most consistent results.

I. INTRODUCTION

Detecting motion from a series of still frames relies upon computing the derivative of the pixel intensities over time. This operates on the assumption that a large portion of the image will have a stationary background, where the pixel intensities minimally change between frames. Motion is detected in areas where the pixel intensities drastically changed very quickly, as it is assumed to be an object moving across the frame. The performance of this detection method is reliant upon two things: the elimination of noise, and the threshold derivative value in which the pixel is considered “in motion”.

In this report, different parameters for noise filtering and masking thresholds are tested to determine the best configuration for motion detection in three different scenes.

II. DESCRIPTION OF ALGORITHMS

A. Motion Detection Pipeline

The motion detection pipeline uses a sequence of frames captured from a stationary camera in order to identify moving objects by analyzing changes in pixel

intensity between time-adjacent frames. The first step in this pipeline is to load and convert each image to grayscale. Optionally, each frame is then smoothed with either a box filter or Gaussian filter. Next, temporal derivatives of each frame in the sequence are calculated. The absolute value of the temporal derivatives is then compared against a threshold value to create a mask, outlining the objects in motion within the sequence. Finally, this mask is applied to the original sequence of images, highlighting the objects in motion and blacking out the stationary background/objects.

B. Temporal Derivative Filter

This is the simplest way of detecting motion between frames. The algorithm simply calculates the derivative value for each pixel by averaging the negative intensity (I) of the pixel in the previous frame and its intensity in the next frame. As shown in the following equation:

$$d = 0.5(-I_{t-1} + I_{t+1})$$

Where I_{t-1} and I_{t+1} represent pixel intensities in the previous and next frames, respectively. This filter is then applied to each frame in the sequence of images. The edge frames (first and last frames) were not padded, rather the change in pixel intensity was averaged between the current frame and next or previous frame, respectively. The computed derivative is close to zero if the pixel is stationary, and will show a spike if the pixel has changes during this frame.

C. Box Filter

The temporal derivative filter does not include any smoothing for the image, so in our algorithm it is combined with a spatial smoothing filter to reduce noise. One way of doing this is with a box filter, which sets each pixel to the average intensity of all pixels surrounding in in a box with a specified size ($N \times N$), where N is an odd integer. The kernel would look as follows:

$$I = \frac{1}{N^2} \begin{bmatrix} 1 & \dots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 1 \end{bmatrix}$$

Larger values of N provide a higher noise reduction, at the cost of blurring the image. This filter was applied using convolution with reflection padding on each frame in the dataset. In reflection padding, pixel edges in a frame are mirrored outward to create a boarder.

D. 2D Spatial Gaussian Filter

A gaussian filter works similarly to the box filter, except it puts more weight on pixels that are closer to the target. The pixel weights follow a Gaussian distribution, defined by the parameter σ . Higher σ values correspond to a wider blur radius, with each weight calculated by the following equation:

$$G_i = \alpha e^{-\frac{\left(i - \frac{N-1}{2}\right)^2}{2\sigma^2}}$$

Where α is a scaling factor that ensures $\sum G_i = 1$. A sample gaussian kernel could look like the following:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This filter was also applied to each image using the convolution operation with reflection padding.

E. 1D Derivative of Gaussian (DoG) Filter

The 1D DoG filter combines the concepts from the spatial gaussian and temporal derivative filter to smooth frames and detect motion in one step. Instead of smoothing frames by spatially adjacent pixels, this filter smooths the derivative output by computing a weighted difference in intensity within a window of past and future frames [1]. Again, the distribution of weights is determined by a standard deviation parameter we will call t_σ . The weights are calculated with the following equation:

$$G(x) = e^{\frac{-x^2}{2t_\sigma^2}}$$

With its derivative being:

$$G'(x) = -\frac{x}{t_\sigma^2} e^{\frac{-x^2}{2t_\sigma^2}}$$

Where x is the position relative to the “center” pixel. In this implementation, the size of the kernel depends on t_σ , with a higher t_σ , representing a wider kernel. In specific, the width of the kernel increases by 6 per t_σ value. For example, a $t_\sigma = 1$ will result in a kernel size of 7. This adjustable kernel size was implemented due to minimal difference between weights of an extremely small and large t_σ value when the width of the kernel is

constant/small. A sample 1D gaussian kernel may look like:

$$0.5[-1, -8, -18, 0, 18, 8, 1]$$

This filter was also applied to each image using the convolution operation with reflection padding.

F. Threshold Selection & Mask Creation

Threshold selection plays a key role in distinguishing between motion and noise within the temporal derivative frames. A threshold that is too low would result in excessive noise reduction, removing aspects of the true motion between frames, comparatively, a threshold that is too large would not remove enough noise, causing the final sequence of images to be noisy. Furthermore, accurate threshold values vary by frame, as each frame as differing pixel intensities. In order to address these challenges, a threshold selection strategy was created for each individual frame.

Using the assumption that most background pixels in a frame change very little with respect to time, allows the noise in the differentiated images to be modeled as Gaussian with zero mean. Therefore, a threshold can be modeled as an arbitrary number of standard deviations of the differentiated frames. The threshold was then modeled to be:

$$threshold = n * \sigma_{df}$$

Where n is an arbitrary constant scalar and σ_{df} is the standard deviation of a differentiated frame. Note that this requires a mask to be created for and applied to each frame in the sequence. Thus, the following formula was applied to each pixel in differentiated frames to create a binary mask:

$$mask = \begin{cases} 1, & abs(I_{pixel}) > threshold \\ 0, & otherwise \end{cases}$$

III. EXPERIMENTS

For consistency, each experiment was run with the “EnterExitCrossingPaths2cor” and “Office” dataset.

A. Spatial Smoothing

The first experiment tests the effectiveness of applying different 2D spatial smoothing filters before motion detection with the temporal derivative. Initially, a simple box filter is tested with sizes 3x3 and 5x5. These results are compared to a 2D gaussian filter with three different S_σ values: 0.1, 1, and 10. The temporal derivative filter is applied as described in Section II-B. The threshold derivative value to mask a pixel in a frame is set equal to three times the standard deviation of all pixel intensities

in the differentiated frame. This value seemed to do a good job of accentuating the difference in noise between the different filters.

B. Temporal Smoothing

The second experiment evaluates the effectiveness of the simple temporal derivative filter described in *Section II-B* and compares it with the 1D DoG filter described in *Section II-E*. Having no smoothing or parameters, the simple derivative filter served as a baseline. The DoG filter was tested using three distinct standard deviations, t_σ : 0.25, 1, and 10 to analyze how temporal smoothing effects motion detect and noise reduction across frames. Each of these t_σ values were used in the full motion detection pipeline using the threshold selection strategy proposed in *Section II-F*, with an n value of 3, and were visually compared against one another.

C. Threshold Selection

The third and final experiment evaluates different threshold values and methods. The threshold selection strategy proposed in *Section II-F*, with three different values chosen for n , was compared with three chosen fixed threshold values. The following values were used for n : 1, 2, and 3 (representing the three common confidence intervals) and the following fixed threshold values: 1, 5, and 25 were chosen. These different values and methods were each used in the full motion detection pipeline using a 3x3 box filter for smoothing and a basic temporal derivative filter. The resulting frames were visually compared against one another.

IV. RESULTS

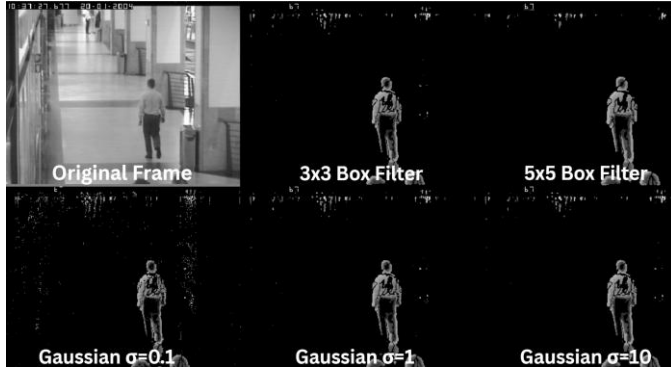


Figure 1: Comparison of Spatial Smoothing Filters

Figure demonstrates the difference between the five different spatial smoothing filters used during testing. Comparing the 3x3 to the 5x5 box filters, we see a marginal decrease in noise. Gaussian filters with small S_σ values (<0.5) underperform the box filters when it comes to removing noise. As the S_σ parameter is increased, it becomes more effective at reducing noise, at the cost of

more blur being added to the image, making it more difficult to detect smaller moving objects and edges.

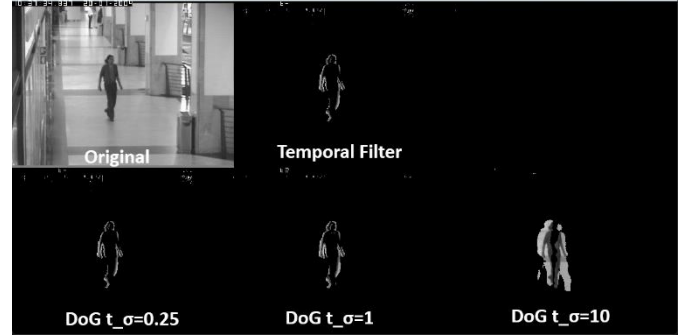


Figure 2: Temporal Smoothing Results - EnterExitCrossingPaths2cor dataset

Figure 2 shows results of the temporal smoothing filters as well as the original frame. Specifically, the basic temporal filtered from *Section II-B* is shown in the top middle box and the 1D DoG filter discussed in *Section II-E* with three different t_σ values are shown in the bottom row. Note that there is an empty frame in the top right corner due to the software used to display each trial simultaneously. Visually inspecting the results in Figure 5, a couple of key observations can be noted. Firstly, compared to the results of the DoG experiments, the temporal derivative filter retains all noise available. However, with increasing the t_σ values, the DoG filter decreases the amount of noise present in the frames. This can best be seen in the difference in the background noise of the top/top right frames where $t_\sigma = 1$ and $t_\sigma = 10$. However, as t_σ increases, the blur of image increases. More specifically, the temporal blur of the frames increase. This can be seen in the difference between the outline of the walking man in the $t_\sigma = 1$ and $t_\sigma = 10$ frames. In the $t_\sigma = 1$ frame, the walking man as a crisp/tight outline, however, in the $t_\sigma = 10$ frame, the outline of the man is very loose and includes pixels in the background which are not necessarily changing. Additionally, when the series of frames are played sequentially, the $t_\sigma = 10$ frames have a “broad” border of motion around the men who walk, indicating their previous and future movements. This is most likely due to the adjusting kernel size implemented in the DoG filter. With a larger kernel size, the time window of previous and future frames is increased. This causes more frames (and the derivative of those frames) to be taken into account when the masks are being created, thereby creating a “bubble” of motion around each moving object which represents its previous and future movements.

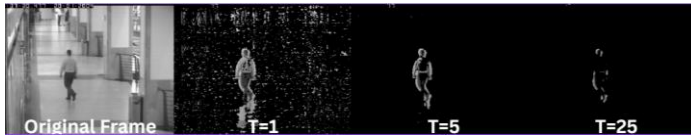


Figure 3: Fixed Threshold Results

Selecting an appropriate threshold value for masking is just as important to the performance of the motion detection algorithm. Figure 3 shows the results of using various fixed threshold values on one frame. The higher the fixed threshold value is, the less likely it will be that the algorithm is able to correctly identify the whole moving (as shown where $T=25$). Selecting a threshold value too low ($T=1$) leads to a large number of false positives, resulting in a noisy mask.



Figure 4: Dynamic Threshold Results



Figure 5: Dynamic Threshold - No Motion

The exact threshold that will perform best will vary from scene to scene and even frame to frame, so a better way of selecting the value may be with an algorithm based on the data we know about the frame. In this case, setting the threshold proportionally to the standard deviation of the frame leads to more consistent performance across the

different datasets. A value of $n=3$ tended to provide the best balance between eliminating noise while still outlining all moving objects. One potential issue we noticed with this method is a spike in noise when there are no moving objects on screen. This does not occur in the “EnterExitCrossingPaths2cor” dataset but is shown in Figure 5 with the “Office” dataset. The effect is less pronounced at higher n values, but still notable at our selected $n=3$.

V. CONCLUSION

In conclusion, there are several methods and parameters that need to be considered when creating a motion detection algorithm. The method that works the best will vary depending on the use case. Using a 2D spatial smoothing filter was generally less effective at removing noise from the resulting mask, but it correctly masks much more of the moving person in comparison to the temporal filter, which only highlights the outline of the moving object in many cases. Despite their better noise reduction qualities, temporal filters are susceptible to ghosting artifacts. Threshold selection also depends on the application: In stationary scenes where movement is less common, a fixed threshold value may work well. In highly dynamic scenes where you have changing lighting and/or multiple moving objects at all times, it will be beneficial to calculate a new threshold for each frame.

REFERENCES

- [1] **Torralba, A., Isola, P., & Freeman, W. (2024). Foundations of computer vision. MIT Press. <https://mitpress.mit.edu/9780262048972/foundations-of-computer-vision>**